BANTADS: suposições da equipe

DS152, Desenvolvimento de Aplicações Corporativas (UFPR, TADS, 2026/2)

O enunciado pede que toda suposição que não esteja definida nele seja documentada (seção 5). Este documento lista o que a equipe precisou decidir, o que foi adotado e onde isso aparece no sistema.

1. Contrato e testes

1. O enunciado cita a aplicação de teste `test_dac`. Usamos a suíte do BANTADS 2026/2 (`razeranthom/test_bantads`) e o `swagger.yaml` (BANTADS API 2.0.0) como contrato dos endpoints: caminhos, corpos, códigos de resposta e o formato de erro `{ status, erro, mensagem }`. O 401 segue o enunciado: `{ auth, message }`.
2. O API Gateway tem um endpoint que não está no swagger: `GET /cep/{cep}`. A tela de autocadastro usa esse endpoint para preencher o endereço, e o Gateway consulta o serviço público ViaCEP. O front não chama o ViaCEP direto, porque só pode conversar com o Gateway. Se o ViaCEP não responder em 5 segundos, a tela avisa e o endereço é digitado à mão.
3. Cada microsserviço tem um `POST /reset` interno, chamado só pelo Gateway durante o `POST /reboot`. Os microsserviços não publicam porta no docker-compose, então esse endpoint não é alcançado de fora.

2. Autocadastro e clientes (R1, R8, R9, R10)

4. O back-end não confere o dígito verificador do CPF, só os 11 dígitos, porque a aplicação de teste gera CPFs sem dígito verificador válido. Quem confere o dígito é a tela de autocadastro.
5. "Não pode se autocadastrar mais de uma vez" vale para qualquer estado da solicitação. Um CPF que já tem solicitação Pendente, Aprovada ou Não aprovada recebe 409 numa nova tentativa. O e-mail segue a mesma regra, sem diferenciar maiúsculas de minúsculas.
6. O autocadastro exige CEP com 8 dígitos e UF com 2 letras. O complemento é opcional.
7. O enunciado deixa o endereço dos clientes pré-cadastrados por nossa conta. Usamos endereços de Curitiba/PR e escolhemos também os telefones de clientes e gerentes.
8. A senha gerada na aprovação (R9) tem 8 caracteres, com letras e números. Ficaram de fora os caracteres que se confundem na leitura (0 e O, 1, l e I).
9. Na aprovação, o cliente vai para o gerente ativo com menos clientes. No empate fica o de menor CPF, que é a ordem em que o MS Gerente lista os ativos.
10. Aprovar de novo uma solicitação que já foi processada termina o job em FALHA.
11. A recusa (R10) exige o motivo e só vale para solicitação Pendente.

3. Contas e operações (R3 a R7)

12. O número da conta tem 4 dígitos sorteados. Se o número já existir, sorteia outro.

13. A conta nova começa com saldo 0,00. O evento `Criado` guarda também o salário e um limite (metade do salário, para salário a partir de R$ 2.000,00), mas o limite não entra em nenhuma operação: saque e transferência exigem saldo.
14. Os valores trafegam como texto, com ponto e até duas casas (`"150.00"`), com no máximo 15 dígitos antes do ponto. Fora desse formato, ou com valor zero, a operação é recusada.
15. Na concorrência (optimistic locking), a operação que perde a disputa de versão refaz o replay e revalida até 3 vezes. Se perder as três, responde 409 e o cliente repete a operação.
16. Transferência para a própria conta ou para conta que não existe responde 422.
17. O gerente consulta a conta e o extrato de qualquer cliente, porque a aplicação de teste exige. Depositar, sacar e transferir é só do cliente dono da conta: para o gerente, 403.
18. No extrato, sem datas vale o período dos últimos 30 dias. Intervalo acima de 365 dias, ou fim antes do início, responde 422. As datas usam o fuso `America/Sao_Paulo` (variável `TZ` dos contêineres).
19. A sincronização do lado de consulta é feita pela fila `ms.conta.events`. Além disso, a projeção se recupera sozinha lendo o event store em dois casos: quando chega um evento com lacuna de versão (uma mensagem anterior se perdeu) e quando o cliente consulta a própria conta por `GET /clientes/{cpf}/conta` ou o extrato, para ele enxergar a operação que acabou de fazer. `GET /contas/{numero}` devolve o que já foi projetado.

4. Gerentes (R12 a R15)

20. Na R13, os empates que o enunciado não cobre são resolvidos pela ordem de CPF dos gerentes e, entre contas de mesmo saldo, pelo menor número de conta.
21. Na R13 o telefone é obrigatório, como no swagger. Nome e e-mail aceitam até 100 caracteres e telefone até 20.
22. Na R14 só nome e telefone mudam. Enviar e-mail ou CPF diferentes dos cadastrados responde 400.
23. Na R15, todas as contas do gerente removido vão para o gerente ativo que tem menos clientes naquele momento (no empate, o de menor CPF), e cada cliente transferido recebe um e-mail. O gerente removido fica Inativo, não é apagado, e deixa de aparecer na listagem.
24. O mesmo usuário pode ter mais de um login aberto ao mesmo tempo: a aplicação de teste exige que o token antigo continue valendo depois de um novo login. A chave reversa `sessao:cpf:<cpf>` aponta para o login mais recente. Na R15 o Orquestrador apaga a chave reversa, a sessão apontada por ela e as outras sessões do mesmo CPF.

5. Autenticação e Redis

25. O `exp` do JWT é de 8 horas (variável `JWT_EXPIRES_IN`). Sem `JWT_SECRET` no ambiente o API Gateway não sobe.
26. O token revogado no logout fica na chave `sessao:revogado:<jti>`, com TTL igual ao tempo que faltava para o `exp`.
27. O navegador guarda no localStorage só o token, o perfil e os dados do usuário devolvidos no login (CPF, nome e e-mail).

28. O cache guarda só dados cadastrais (`cache:cliente:<cpf>` e `cache:gerente:<cpf>`, 5 minutos). Nada que vem do MS Conta é guardado.

29. A senha gerada na R9 e a senha informada na R13 ficam só na memória do Orquestrador e nas mensagens que precisam delas. Se o Orquestrador reiniciar no meio de uma SAGA, o passo que dependia da senha falha e a SAGA é compensada.

6. Mensageria e SAGAs

30. "3 tentativas com intervalo de 5 segundos" foi implementado como 3 novas tentativas depois da primeira entrega, com a retentativa do Spring no consumidor. São 4 execuções em cerca de 15 segundos e, depois, a DLQ, dentro dos 15 a 20 segundos citados no enunciado.

31. Falha de regra de negócio (CPF ou e-mail repetido, último gerente ativo) não passa pela retentativa: o microsserviço responde `FALHA` na hora e o Orquestrador compensa. A retentativa e a DLQ cobrem os casos em que o consumidor não consegue responder, como banco fora do ar ou erro inesperado.

32. Para o timeout de 30 segundos, o Orquestrador guarda o prazo de cada passo no Redis (`saga:timeouts`) e confere em intervalos curtos. Resposta, DLQ e timeout do mesmo passo disputam a chave `saga:<id>:etapa:<n>`, e só o primeiro sinal é tratado.

33. Cada microsserviço guarda a resposta de cada comando por (sagaId, tipo) e a reenvia se o comando chegar de novo. No PostgreSQL é a tabela `saga_idempotencia` de cada schema e no MongoDB a coleção `comandos_processados`.

34. A fila `ms.conta.events.dlq` não tem consumidor. O reprocessamento é manual, pelo console do RabbitMQ.

35. O relatório (R16) é um job assíncrono montado pelo próprio Gateway, por API Composition e sem SAGA. O resultado inline é `{ clientes: [...] }`.

7. Dados e infraestrutura

36. As tabelas são criadas pelas migrações Flyway de cada serviço (`src/main/resources/db/migration`). Os dados pré-cadastrados são gravados pelo próprio serviço, na primeira subida com o banco vazio e a cada `POST /reboot` (classes `ResetService` e `SeedUsuarios`). No MS Conta o seed grava os eventos e a projeção. Não há script SQL de carga separado.

37. O docker-compose publica as portas 4200 (front), 3000 (API Gateway) e 15672 (console do RabbitMQ). As portas 5432, 27017, 6379 e 5672 também ficam publicadas, para as ferramentas de banco abertas na defesa e para rodar um serviço fora do Docker durante o desenvolvimento.

38. Todos os contêineres têm `mem_limit`, e a soma fica em cerca de 5 GB. O Redis ficou com 256 MB (o enunciado sugere 128 MB na coluna de 8 GB) e `--maxmemory 100mb`.

39. Os e-mails saem pelo SMTP do Gmail (porta 587, STARTTLS), com `GMAIL_USERNAME` e `GMAIL_APP_PASSWORD` lidos do `.env`. As variáveis `SMTP_HOST` e `SMTP_PORT` permitem apontar para outro servidor em teste.